

PROJECT REPORT

Personal Expense Tracker with Smart Budget Alerts

Java Command-Line Application

Student Name	Pranshu Mittal
Registration No.	25BAI11550
Programme	CSE (AI/ML)
Institute	VIT Bhopal University
Technology	Java 17+
Repository	github.com/PranshuMittal7/Vityarthy_Java

Submitted as part of the Vityarthy Java project requirements

Report Contents

- 1. Introduction
- 2. Problem Statement
- 3. Functional Requirements
- 4. Non-functional Requirements
- 5. System Architecture
- 6. Design Diagrams
- 7. Design Decisions & Rationale
- 8. Implementation Details
- 9. Screenshots / Results
- 10. Testing Approach
- 11. Challenges Faced
- 12. Learnings & Key Takeaways
- 13. Future Enhancements
- 14. References

1. Introduction

Personal Expense Tracker with Smart Budget Alerts is a Java 17+ command-line application designed to record income and expenses, organize spending by category, persist transaction data locally, generate summary reports, and provide active budget notifications. The application is intentionally lightweight: it uses the Java standard library and a plain CSV file instead of a database or external service.

The project demonstrates practical use of core Java concepts including classes and objects, enumerations, collections, file I/O, exception handling, synchronization, and multithreading. The main user interaction is handled by a menu-driven console interface, while a separate daemon thread periodically checks category spending against predefined limits.

The central idea is to make expense tracking more active. Instead of requiring the user to generate a report before noticing high spending, the BudgetMonitor checks spending concurrently and emits a warning at 80% of a category budget and an alert at 100%.

Project scope includes transaction CRUD operations, category filtering, CSV persistence, budget monitoring, reporting, validation, and corrupted-file handling. The project does not include a graphical interface, authentication, cloud synchronization, bank integration, or a relational database.

2. Problem Statement

Traditional personal expense records are often passive. A user may enter transactions in a notebook, spreadsheet, or application, but the information may only become useful after the user deliberately reviews it. This creates a gap between spending and feedback.

The project addresses this gap by combining ordinary transaction management with continuous category-level budget monitoring. When spending approaches or crosses a predefined limit, the monitoring thread prints a notification independently of the main menu flow.

Proposed solution: provide a lightweight command-line tracker that records transactions, stores them locally, calculates totals, compares expense categories with budgets, and reports threshold breaches in real time.

Target users: students, early-career professionals, and other individuals who want a simple local tool for monitoring routine personal spending.

Scope limitations: the current application is single-user and local. It does not provide authentication, cloud storage, automatic bank imports, multiple currencies, forecasting, investment tracking, or tax calculations.

3. Functional Requirements

ID	Requirement	Description
FR1	Add transactions	The system shall accept income or expense transactions with amount, category, description and date.
FR2	View transactions	The system shall display all stored transactions.
FR3	Delete transactions	The system shall delete a transaction using its generated ID.
FR4	Filter by category	The system shall display transactions belonging to a selected category.
FR5	Budget monitoring	The system shall monitor expense totals by category in a background thread and issue threshold notifications.
FR6	Persistence	The system shall load transactions from transactions.csv at startup and save them on exit.
FR7	Reporting	The system shall generate total income, total expense, net balance, category spending and a monthly category report.
FR8	Report file	The generated report shall also be written to report.txt.
FR9	Validation	The system shall reject invalid amounts, invalid menu/category choices and blank descriptions.
FR10	Corrupt-data handling	Malformed CSV records shall be skipped individually while valid records continue to load.

4. Non-functional Requirements

Attribute	Requirement
Usability	Menu-driven console interface with clear prompts and feedback.
Reliability	Invalid input and malformed saved records should not terminate the application unexpectedly.
Maintainability	Responsibilities are separated across focused classes such as ExpenseTracker, FileManager, BudgetMonitor.
Concurrency safety	Shared transaction access is synchronized because the main thread and monitoring thread access the same data.
Portability	The project uses Java standard-library features and does not require third-party dependencies.
Performance	For the intended small personal dataset, in-memory ArrayList processing and periodic checks are lightweight.
Data transparency	Transactions are stored as readable CSV records in the local project directory.
Security / privacy	The application has no network component or remote account system; local file protection is therefore the responsibility of the user.

5. System Architecture

The application follows a small modular architecture centered on the Main class. Main handles user interaction and coordinates the other components. ExpenseTracker owns the in-memory transaction collection and provides synchronized operations. BudgetMonitor runs independently as a daemon thread and reads spending through synchronized tracker methods. FileManager handles CSV persistence and report-file output. ReportGenerator calculates summary and month-by-category reporting data. Transaction and Category represent the domain data.

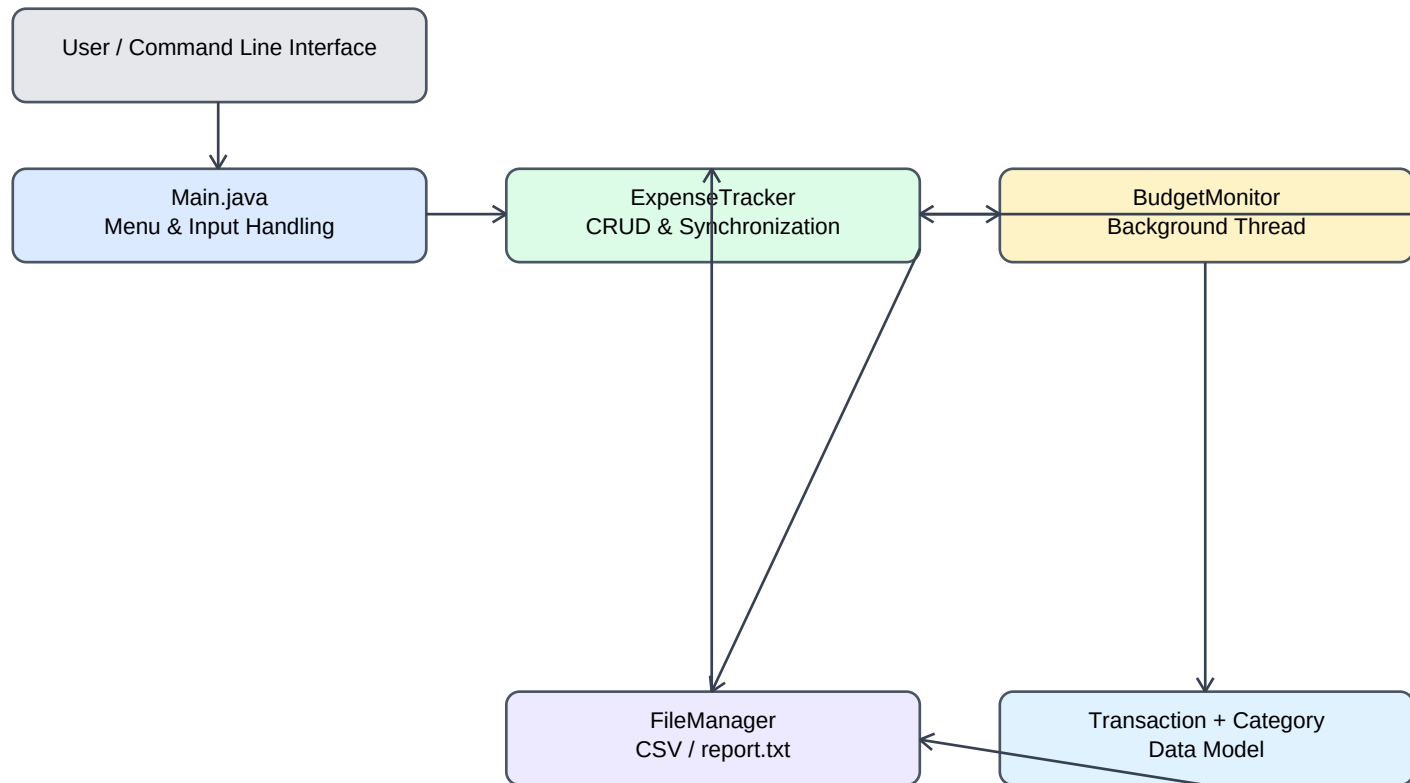


Figure 1: High-level system architecture

Data flow is primarily local: user input enters through Main, transaction operations update ExpenseTracker, reports read from the tracker, and FileManager persists the data. BudgetMonitor observes the tracker concurrently. No external API, server, or database is required.

6. Design Diagrams

6.1 Use Case Diagram

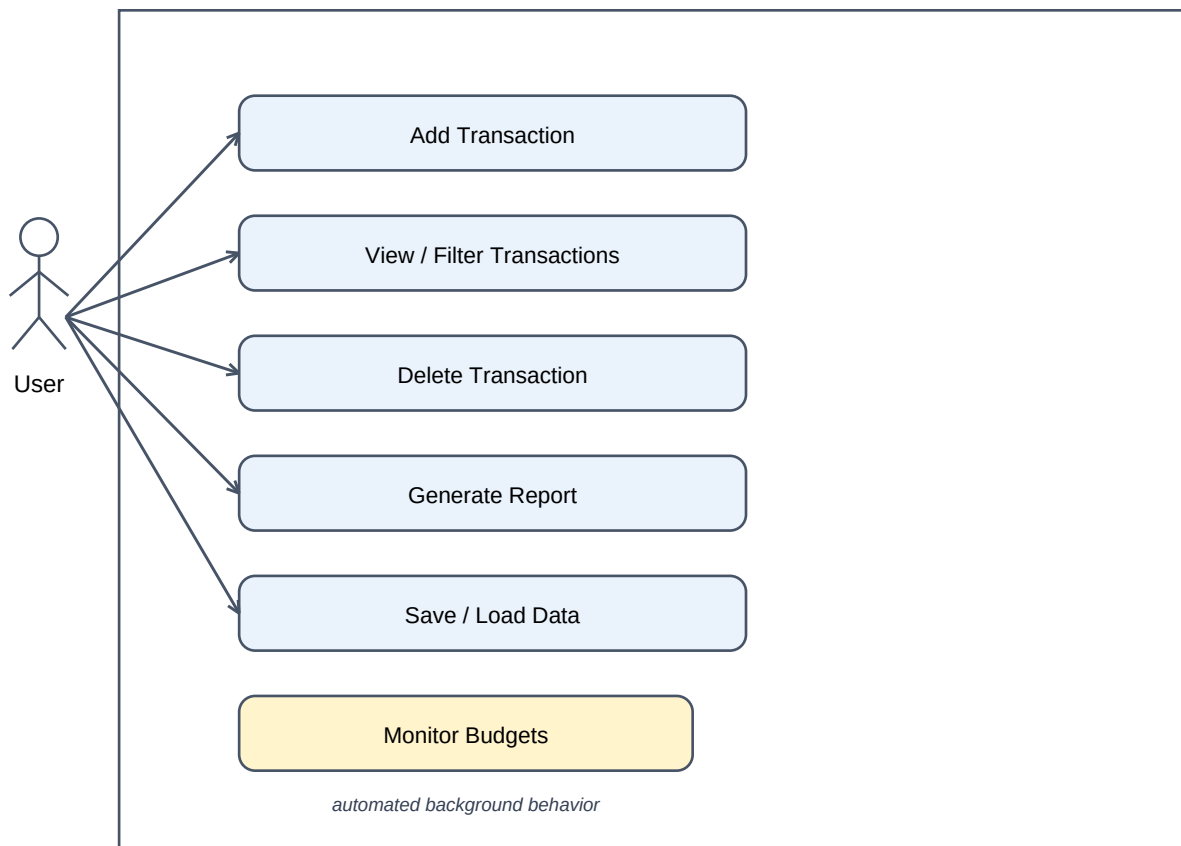


Figure 2: Use case view of the application

The user performs transaction and reporting operations. Budget monitoring is an automated behavior carried out by the application while the user continues interacting with the menu.

6.2 Workflow Diagram

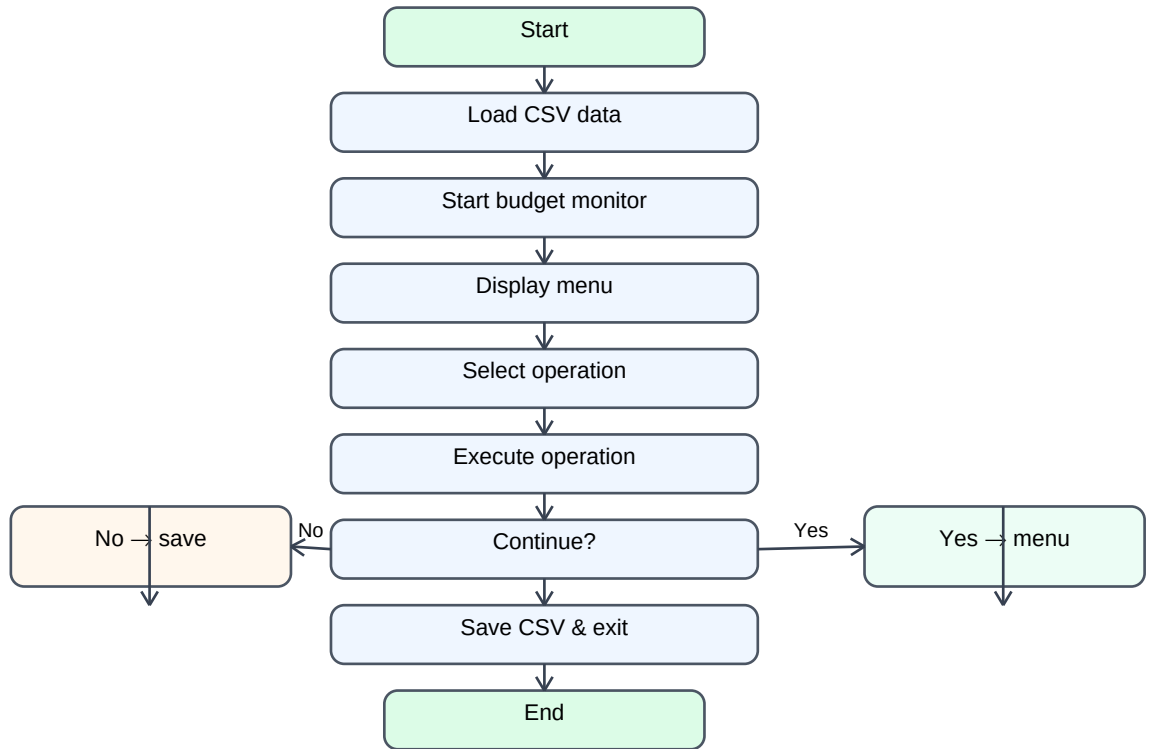


Figure 3: Main application workflow

6.3 Sequence Diagram

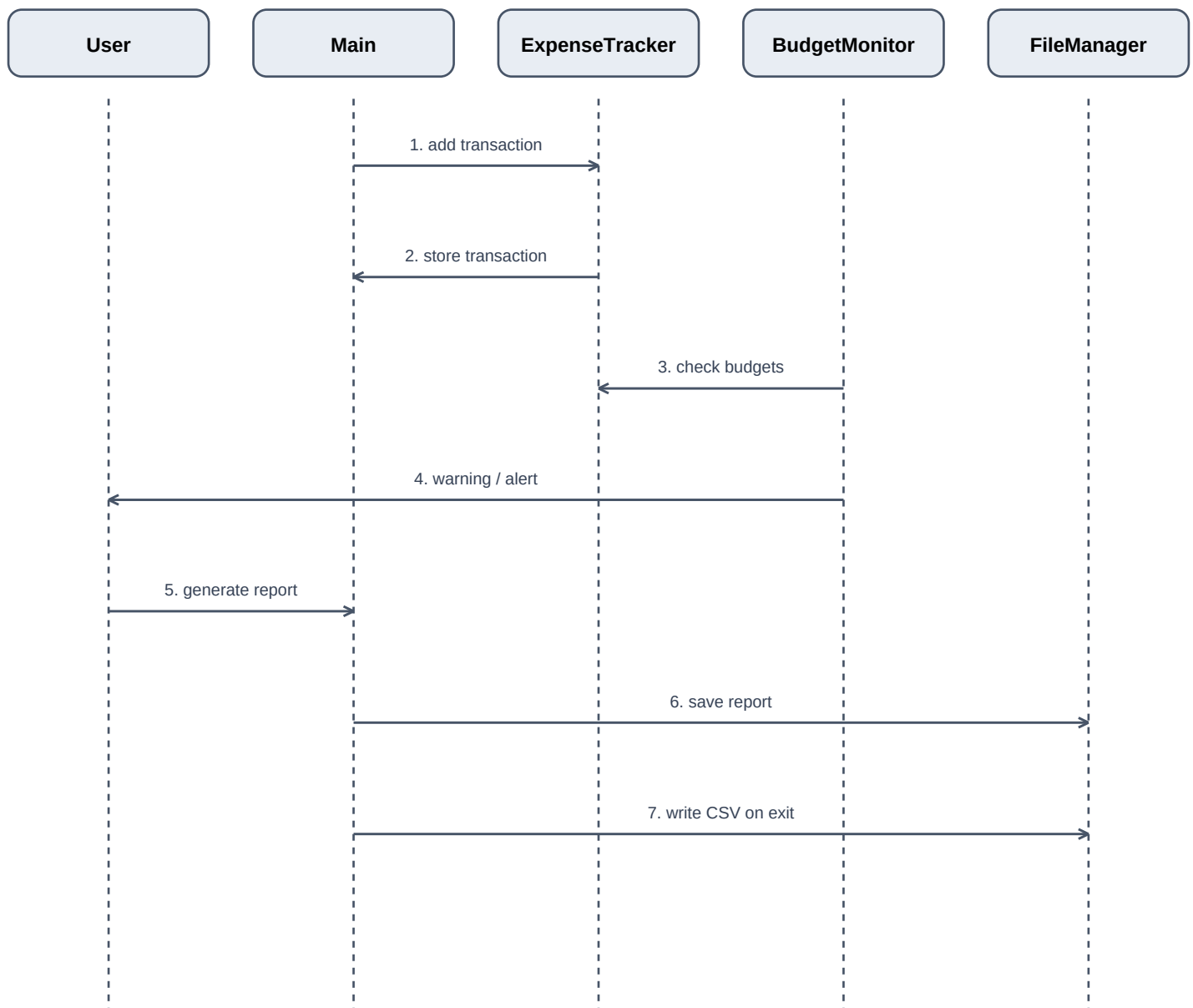
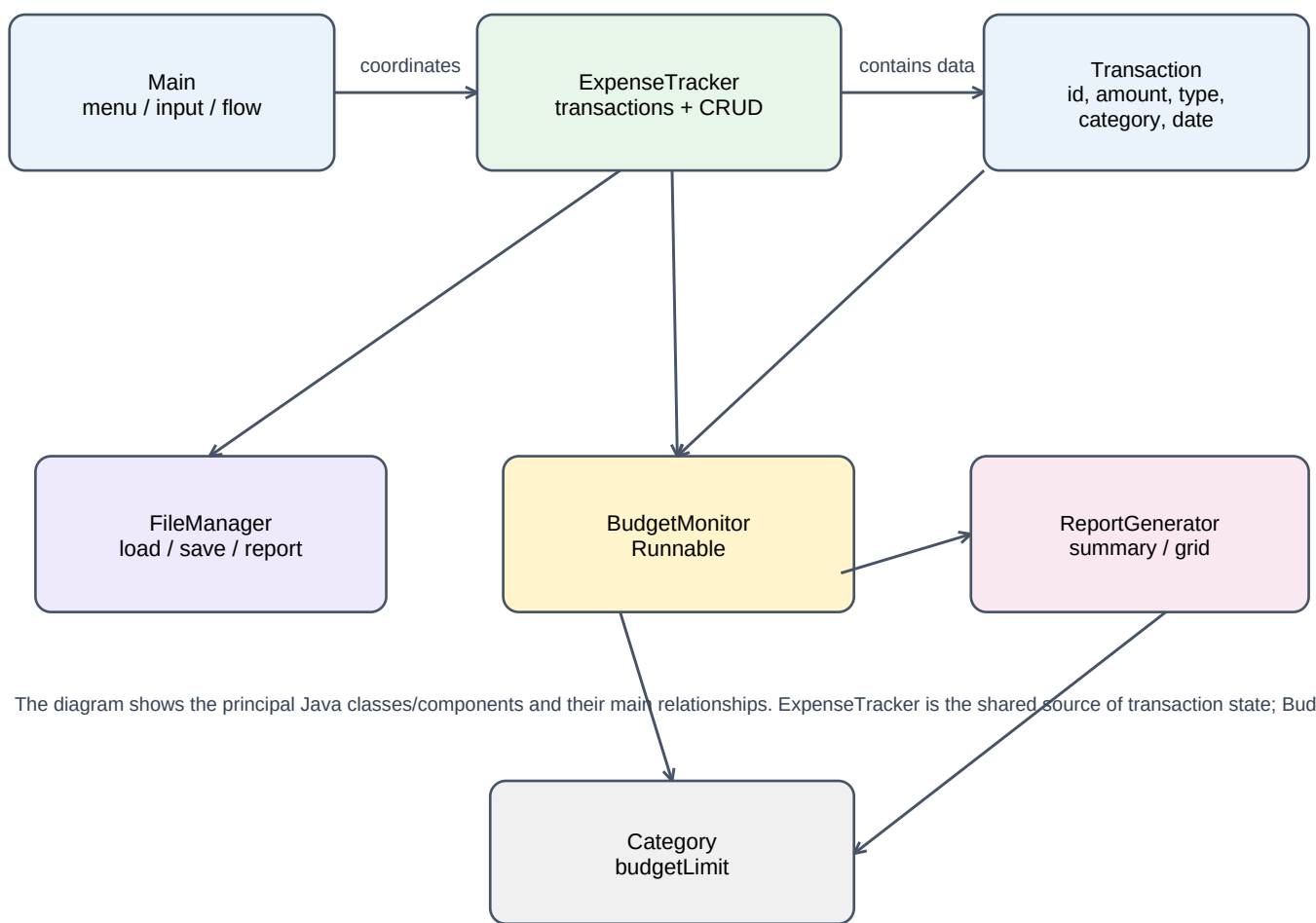


Figure 4: Representative interaction sequence

The sequence combines the important runtime interactions: transaction creation, synchronized storage, background budget checking, report generation and file persistence during exit.

6.4 Class / Component Diagram



The diagram shows the principal Java classes/components and their main relationships. ExpenseTracker is the shared source of transaction state; BudgetMonitor

Figure 5: Class/component relationships

6.5 ER Diagram / Logical Storage Model



Note: The implementation uses a flat CSV file rather than a relational database. This is a logical ER-style model for the stored records.

Figure 6: Logical data model for CSV storage

The project does not use a relational database. The diagram therefore represents the structure and relationship of the records stored in transactions.csv rather than claiming that database tables exist. Each transaction stores an ID, date, type, category, amount and description; Category provides the predefined category and budget limit used by the application.

7. Design Decisions & Rationale

- **Command-line interface:** chosen to keep the project small and focused on Java programming concepts rather than UI frameworks. It also keeps the application dependency-free.
- **CSV persistence:** selected because transaction data is simple, portable and human-readable. It avoids database setup for a small single-user application.
- **ArrayList for transactions:** suitable for a lightweight tracker where records are iterated for totals, filtering and reporting.
- **Enum for categories:** Category centralizes the fixed set of categories and their budget limits, reducing invalid category values.
- **Synchronized tracker methods:** the monitor thread and main thread can access the same transaction collection, so synchronized methods reduce the risk of concurrent modification/race conditions.
- **Daemon monitoring thread:** BudgetMonitor performs periodic checks without blocking the main menu. The thread is stopped explicitly when the user exits.
- **Custom exceptions:** InvalidTransactionException and FileParseException make recoverable validation and parsing failures explicit. BudgetExceededException is defined as an unchecked exception type for budget breaches, although the current BudgetMonitor reports breaches directly rather than throwing it.
- **Report generation:** ReportGenerator separates reporting calculations and formatting from menu/input logic, improving organization and maintainability.

Important implementation note: the code labels category limits as monthly budgets and provides a monthly reporting grid, but `getTotalSpentInCategory()` totals all loaded expenses in a category without filtering by month. Therefore, the current budget warning behaves like a cumulative category limit over the stored dataset rather than a true calendar-month-reset budget. This is a limitation of the current implementation and a clear future enhancement.

8. Implementation Details

- **Main.java:** defines the application entry point, menu loop, input helpers, loading/saving flow and coordination of components. It starts BudgetMonitor before entering the menu and stops it on exit.
- **ExpenseTracker.java:** maintains the ArrayList of transactions. It implements add, restore, delete, list, category/type filtering, grouping, total income, total expense and category spending methods. Public methods are synchronized.
- **Transaction.java:** is the data model. It stores ID, amount, category, type, description and date. New IDs are generated from UUID values and shortened to eight characters. The class also converts records to and from CSV.
- **Category.java:** defines FOOD, RENT, ENTERTAINMENT, TRANSPORT, UTILITIES, SALARY and OTHER. Expense categories have fixed budget limits; SALARY has a zero limit and is excluded from budget reporting.
- **FileManager.java:** uses BufferedReader/BufferedWriter and standard file APIs to load and save CSV records and write report.txt. A malformed line is skipped rather than stopping the entire load.
- **BudgetMonitor.java:** implements Runnable. A daemon worker checks each budget every 5,000 ms. HashSet objects prevent repeated warning/alert messages for the same category during one run.
- **ReportGenerator.java:** calculates total summaries through the tracker and builds a 7-category by 12-month expense grid based on transaction dates.
- **Exception classes:** provide explicit types for validation failures, file parsing failures and budget threshold breaches.

Budget configuration

Category	Limit (■)
FOOD	3,000
RENT	10,000
ENTERTAINMENT	1,500
TRANSPORT	2,000
UTILITIES	2,500
OTHER	1,000
SALARY	0 (not monitored)

The monitor issues a warning at 80% or more of a positive budget limit and an alert at 100% or more. The current interval is 5 seconds.

9. Screenshots / Results

The following result captures were produced from a test run of the uploaded project. The test dataset contained one salary income of █30,000 and four expenses: FOOD █1,200, FOOD █1,500, TRANSPORT █800 and ENTERTAINMENT █500. Total expenses were █4,000 and the resulting balance was █26,000.

```
Test Run: Main Menu

=== Personal Expense Tracker ===

1. Add transaction
2. View all transactions
3. Delete a transaction
4. View transactions by category
5. Generate report
6. Save & Exit
Choose an option:
```

Figure 7: Main menu

```
Test Run: View All Transactions

--- All Transactions (5) ---
[8e7171b6] 2026-09-18 | INCOME   | SALARY       | 30000.00 | Salary
[2dfcf295] 2026-09-18 | EXPENSE      | FOOD         | 1200.00  | Food1
[35db21ff] 2026-09-18 | EXPENSE      | FOOD         | 1500.00  | Food2
[eccd173d] 2026-09-18 | EXPENSE      | TRANSPORT    | 800.00   | Transport
[e7f92ba8] 2026-09-18 | EXPENSE      | ENTERTAINMENT | 500.00   | Fun
```

Figure 8: All transactions

```
Test Run: Category Filter
```

```
--- FOOD (2) ---
```

[2dfcf295]	2026-09-18		EXPENSE		FOOD		1200.00		Food1
[35db21ff]	2026-09-18		EXPENSE		FOOD		1500.00		Food2

Figure 9: Category filtering

```
Test Run: Budget Monitor
```

```
[WARNING] FOOD is at 90% of its budget (2700.00 / 3000.00)
```

Figure 10: Automatic 80% budget warning

```

Test Run: Report Generation

===== Expense Tracker Report =====
Generated: 2026-09-18

Total Income: 30000.00
Total Expense: 4000.00
Net Balance: 26000.00

--- Spend by Category ---
FOOD          2700.00 / 3000.00 (90%)
RENT           0.00 / 10000.00 (0%)
ENTERTAINMENT 500.00 / 1500.00 (33%)
TRANSPORT     800.00 / 2000.00 (40%)
UTILITIES      0.00 / 2500.00 (0%)
OTHER          0.00 / 1000.00 (0%)

--- Monthly Breakdown ---
FOOD: SEP = 2700 | ENTERTAINMENT: SEP = 500 | TRANSPORT: SEP = 800

```

Figure 11: Generated summary report

Additional submitted result

```

=== Personal Expense Tracker ===

1. Add transaction
2. View all transactions
3. Delete a transaction
4. View transactions by category
5. Generate report
6. Save & Exit
Choose an option:
1
Type - 1) Income 2) Expense: 2
Category:
1) FOOD
2) RENT
3) ENTERTAINMENT
4) TRANSPORT
5) UTILITIES
6) SALARY
7) OTHER
Choose: 1
Amount: 12345
Description: Food
Date (YYYY-MM-DD, or leave blank for today):
Added: [691f713e] 2026-09-17 | EXPENSE | FOOD | 12345.00 | Food

1. Add transaction
2. View all transactions
3. Delete a transaction
4. View transactions by category
5. Generate report
6. Save & Exit
Choose an option:

[ALERT] Budget EXCEEDED for FOOD: spent 12345.00 / limit 3000.00 (412%)

[WARNING] FOOD is at 412% of its budget (12345.00 / 3000.00)

```

Figure 12: Budget exceeded alert captured during testing

10. Testing Approach

Testing was performed using manual command-line scenarios against the uploaded source. The objective was to cover normal transaction flow, validation, persistence, budget monitoring, report generation and corrupted-data handling.

ID	Test	Procedure / Expected Result	Status
T1	Compile and launch	Run javac *.java followed by java Main.	Pass
T2	Add valid transaction	Add income and expense records with valid values.	Pass
T3	Negative amount	Enter -100; application rejects the value and asks again.	Pass
T4	Non-numeric amount	Enter abc; application reports invalid number and asks again.	Pass
T5	Invalid date	Enter an invalid date such as 31-02-2026; application falls back to today.	Pass
T6	View all	Display all saved transactions.	Pass
T7	Category filter	Filter by FOOD and verify only FOOD records are shown.	Pass
T8	Budget warning	FOOD spending reaches █2,700, which is 90% of █3,000.	Pass
T9	Report	Generate report and verify totals/category/month values.	Pass
T10	Persistence	Save, restart and verify transactions reload from CSV.	Pass
T11	Corrupt CSV line	Insert malformed record; valid records still load and malformed line is skipped.	Pass
T12	Exit	Choose Save & Exit and verify data file is written.	Pass

```
Test Run: Corrupted CSV Handling
Skipping corrupted line 7: Expected 6 fields, got 3 in line: this,is,broken
Loaded 6 transaction(s) from transactions.csv
```

Figure 13: Corrupted CSV record skipped while valid records load

11. Challenges Faced

The project involves several implementation challenges that are directly tied to the Java concepts demonstrated:

- Keeping shared transaction data safe while the foreground menu and background monitor access it concurrently.
- Handling user input robustly when numeric values, categories or dates are invalid.
- Preserving usable data when a CSV line is malformed instead of failing the complete load operation.
- Separating file persistence, reporting and budget monitoring from the main menu so that the application remains organized.
- Preventing repeated budget notifications by remembering which categories have already crossed the 80% and 100% thresholds during a run.
- Keeping the report output readable in a terminal while also writing the same content to a text file.

Observed limitation: budget limits are described as monthly, but the current spending calculation does not reset by month. This means historical loaded transactions can contribute to the current category total. Implementing month-aware budget calculations would resolve this mismatch.

12. Learnings & Key Takeaways

- Practical use of Java classes, objects, enums and collections in a complete application.
- Using synchronized methods when multiple threads share mutable state.
- Creating and managing a Runnable-based background task and daemon thread.
- Applying checked and unchecked custom exceptions for different categories of failures.
- Reading and writing structured data using Java character I/O streams.
- Designing a simple persistence format with CSV conversion methods.
- Separating application responsibilities into focused classes rather than placing all logic in Main.
- Building reports from raw transaction data using category and date-based aggregation.
- Testing both expected user flows and failure conditions such as invalid input and corrupted storage.

13. Future Enhancements

- Implement true month-aware budget calculations so each category limit resets for a new calendar month.
- Allow users to configure category budgets instead of hard-coding limits in the Category enum.
- Add date-range filters and month-over-month comparison in reports.
- Support recurring transactions such as rent and subscriptions.
- Add export options such as CSV/PDF reports.
- Replace flat-file persistence with SQLite or another embedded database if the dataset grows.
- Add a graphical or web interface for easier interaction.
- Add authentication and separate data files/accounts for multi-user operation.
- Add charts for spending trends and category distribution.
- Add automated unit/integration tests using a Java testing framework.

14. References

1. Project source repository: github.com/PranshuMittal7/Vityarthy_Java
2. Project README.md and statement.md included in the submitted source repository.
3. Java Platform Standard Edition documentation for `java.io`, `java.time`, `java.util` and `java.util.concurrent.atomic` packages.
4. Course/project submission requirements provided for the Vityarthy Java project report.

Conclusion

The Personal Expense Tracker demonstrates how core Java features can be combined into a practical command-line application. Its main differentiator is concurrent budget monitoring, while CSV persistence and report generation provide a complete end-to-end workflow. Testing confirms the main transaction, reporting, persistence, validation, monitoring and corrupted-data paths. The most important current limitation is that category budget calculations are cumulative across stored transactions rather than explicitly month-scoped, which should be addressed in a future version.